

Notes 10: More Classification

Table of contents

Introduction	185
Naïve Bayes Classifier	185
Algorithm	186
Naïve Bayes Example	186
Advantages and Disadvantages of Naïve Bayes	191
Linear Discriminant Analysis	193
LDA for One Predictor	193
LDA with More than One Predictor	196
LDA with Multiple Predictors and Three Classes	197
Advantages and Disadvantages of LDA	199
Support Vector Machines	201
Support Vector Machines–Linear Data	201
A Look into SVM Methodology	202
Support Vector Machines in R	203
Support Vector Machines for Non-Linear Data	209
Advantages and Disadvantages of SVMs	215

Introduction

In this set of notes, we will explore a few more common classification techniques. The first two put some of our new Bayesian knowledge to good use and then we will explore one of the most popular machine learning techniques for classification: support vector machines.

Naïve Bayes Classifier

The naïve Bayes classifier is based on using Bayes' Theorem for assigning posterior probabilities of belonging to a certain class given the data vector. This is not a single algorithm but a family of algorithms where all of them share a common principle: every pair of features being classified is independent of each other. It is highly used in text classification where data contains high dimension (as each word represent one feature in the data). It is used in spam filtering, rating classification, and more. One big advantage of using naïve Bayes is its speed. It is fast and making prediction is easy with high dimension of data.

Assumptions:

- Independence: The observations are conditionally independent of each other, given the class label.
- Continuous variables are normally distributed: If a predictor variable is continuous, then it is assumed to be normally distributed within each class.

- Discrete variables have multinomial distributions: If a predictor variable is discrete, then it is assumed to have a multinomial distribution (a more general case of the binomial) within each class. (This can be adjusted.)
- Variables are equally important: All variables are assumed to contribute equally to the prediction of the class label.
- No missing data: The data should not contain any missing values.

These assumptions are what make this Bayesian classifier “naïve”. If these are not satisfied, more advanced versions of this classifier exist.

Algorithm

We have discussed Bayes’ theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

In this context, we replace these events with the data:

$$P(y_i = k|\mathbf{x}_i) = \frac{P(\mathbf{x}_i|y_i = k)P(y_i = k)}{P(\mathbf{x}_i)}$$

where k is a class in the data for $k = 1, \dots, K$ and $\mathbf{x}_i$ is the i th row of the model matrix or data frame. Written another way, we can say

$$P(y_i = k|x_{i1}, \dots, x_{ip}) = \frac{P(x_{i1}, \dots, x_{ip}|y_i)P(y_i = k)}{P(x_{i1}, \dots, x_{ip})}.$$

Assuming independence, we have that $P(x_{i1}, \dots, x_{ip}|y_i = k) = P(x_{i1}|y_i = k) \times \dots \times P(x_{ip}|y_i = k)$. That gives us

$$P(y_i = k|x_{i1}, \dots, x_{ip}) = \frac{P(x_{i1}|y_i = k) \times \dots \times P(x_{ip}|y_i = k)P(y_i = k)}{P(x_{i1}, \dots, x_{ip})} = \frac{P(y_i = k) \prod_{j=1}^p P(x_{ij}|y_i = k)}{P(x_{i1}, \dots, x_{ip})}.$$

Since the denominator does not have a y_i in it, we can do our proportional trick:

$$P(y_i = k|x_{i1}, \dots, x_{ip}) \propto P(y_i = k) \prod_{j=1}^p P(x_{ij}|y_i = k).$$

Naïve Bayes Example

Let’s try this out on a small example.

Y: Heart Attack	X ₁ : diastolic blood pressure	X ₂ : sex	X ₃ : age
1	92	F	71
1	97	F	64
0	66	F	53
0	83	F	75
0	82	F	69
0	91	F	63
0	77	F	71
1	90	M	57
1	86	M	82
1	96	M	73
0	69	M	86
1	92	M	65
1	84	M	78

Let's focus on the first row. We have, when assuming independence,

$$P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) \propto P(Y = 1)P(X_1 = 92|Y = 1)P(X_2 = F|Y = 1)P(X_3 = 71|Y = 1)$$

These probabilities (or densities in the case of the numeric values) are computed from the data. For the categorical probabilities:

- $P(Y = 1) = 7/13$ since 7 out of the 13 observations in the data set had a Y value of 1.
- $P(X_2 = F|Y = 1)$ is the probability of female given a heart attack, which is $2/7$ since two of the seven people that had a heart attack were female.

For the continuous probabilities, we assume they have a normal distribution. The mean and variance of this normal are going to be the sample mean and sample variance from the class we are conditioning on. Let's call this class k . We call the sample mean conditioned on that class $\hat{\mu}_k$ and we call the sample variance conditioned on that class $\hat{\sigma}_k^2$. So, we get

$$P(X_j = x_{ij}|Y = y_i) = \frac{1}{\sqrt{2\pi\hat{\sigma}_k^2}} \exp\left(-\frac{1}{2\hat{\sigma}_k^2}(x_{i1} - \hat{\mu}_k)^2\right)$$

For $P(X_1 = x_{i1}|Y = 1)$, the data values are 92, 97, 90, 86, 96, 92, and 84. The mean of those values is 91 and the variance is 23. So we have

$$P(X_1 = 92|Y = 1) = \frac{1}{\sqrt{2\pi \cdot 23}} \exp\left(-\frac{1}{2 \cdot 23}(92 - 91)^2\right) = \text{dnorm}(92, 91, \text{sqrt}(23)) = 0.08140.$$

For $P(X_3 = x_{i3}|Y = 1)$, we have the data values 71, 64, 57, 82, 73, 65, and 78. So $\hat{\mu}_k = 70$ and $\hat{\sigma}_k^2 = 74.67$. That gives

$$P(X_3 = 71|Y = 1) = \frac{1}{\sqrt{2\pi \cdot 74.67}} \exp\left(-\frac{1}{2 \cdot 74.67}(71 - 70)^2\right) = \text{dnorm}(71, 70, \text{sqrt}(74.67)) = 0.04586.$$

Putting it all together, we get

$$\begin{aligned} P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) &\propto P(Y = 1)P(X_1 = 92|Y = 1)P(X_2 = F|Y = 1)P(X_3 = 71|Y = 1) \\ &= \frac{7}{13} \times 0.08140 \times \frac{2}{7} \times 0.04586 \\ &= 0.0005743 \end{aligned}$$

Great, but that tells us almost nothing about if this person would be classified as a 0 or a 1. This is not the conditional probability of being in that class, it is only proportion to that probability. We need something to compare it to. We also need to compute $P(Y = 0|X_1 = 92, X_2 = F, X_3 = 71)$.

Data values for $X_1|Y = 0$:

So, we can say $P(Y = 1|X_1 = 92, X_2 = F, X_3 = 71) =$

And $P(Y = 0|X_1 = 92, X_2 = F, X_3 = 71) =$

Naïve Bayes in R

```
library(naivebayes) |> suppressPackageStartupMessages()
library(caret) |> suppressPackageStartupMessages()
heart <- data.frame(y = factor(c(1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1)),
                    x1 = c(92, 97, 66, 83, 82, 91, 77, 90, 86, 96, 69, 92, 84),
                    x2 = c(rep("F", 7), rep("M", 6)),
                    x3 = c(71, 64, 53, 75, 69, 63, 71, 57, 82, 73, 86, 65, 78)
                    )
nb <- naive_bayes(y ~ x1 + x2 + x3, data = heart)
head(predict(nb, type = "prob")) |> round(3)
```

	0	1
[1,]	0.248	0.752
[2,]	0.237	0.763
[3,]	1.000	0.000
[4,]	0.784	0.216
[5,]	0.842	0.158
[6,]	0.307	0.693

```
confusionMatrix(predict(nb, type = "class"), heart$y)$table
```

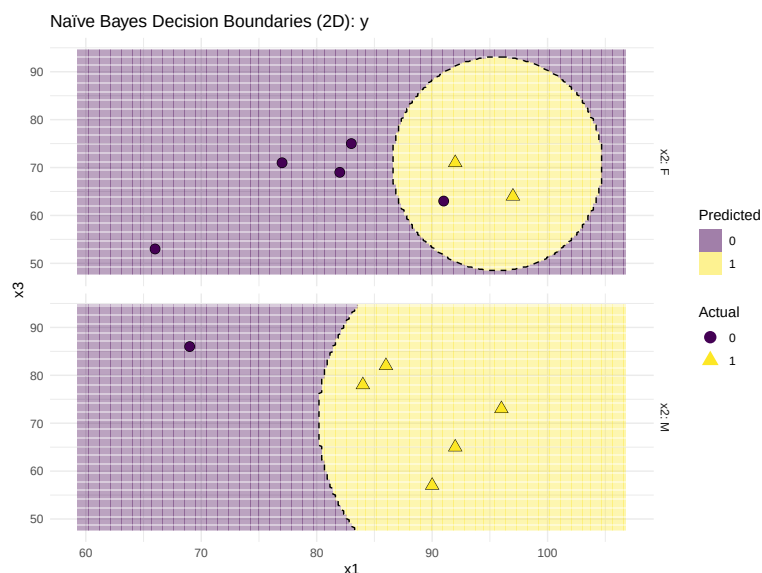
```

      Reference
Prediction 0 1
      0 5 0
      1 1 7

```

In the `math3190package` library, there is a function called `classifier_plot()`, which takes a classifier model (for any of the classifiers we have in this chapter) and returns a plot showing the classification.

```
library(math3190package) |> suppressPackageStartupMessages()
classifier_plot(nb, data = heart)
```



More Classes

We can extend this idea to more than 2 classes. For instance, if a `y` value of 2 indicates the person has had more than 1 heart attack:

```
heart2 <- data.frame(y = factor(c(1, 2, 0, 0, 0, 0, 0, 1, 2, 1, 0, 2, 1)),
  x1 = c(92, 97, 66, 83, 82, 91, 77, 90, 86, 96, 69, 92, 84),
  x2 = c(rep("F", 7), rep("M", 6)),
  x3 = c(71, 64, 53, 75, 69, 63, 71, 57, 82, 73, 86, 65, 78)
)
```

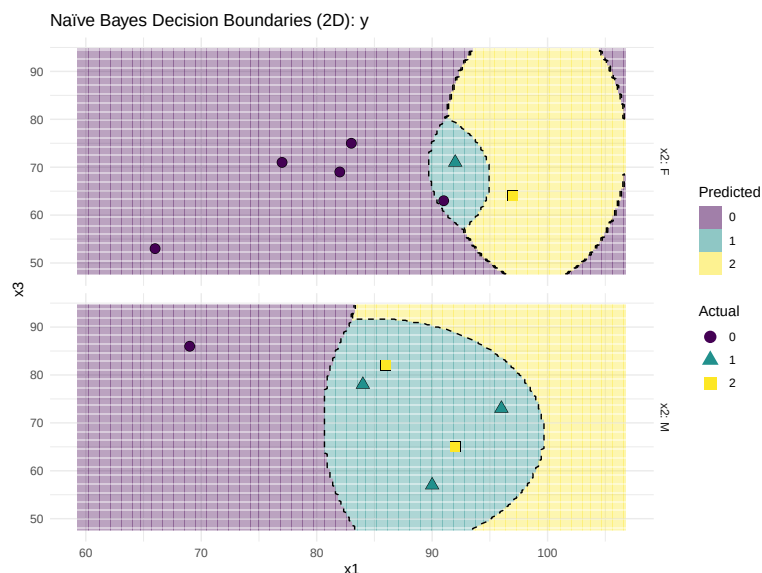
```
nb2 <- naive_bayes(y ~ x1 + x2 + x3, data = heart2)
predict(nb2, type = "prob")[1,]
```

```

      0      1      2
0.2846944 0.3868156 0.3284900

```

```
classifier_plot(nb2, data = heart2)
```



If we are uncomfortable with the assumption that the quantitative variables are normally distributed (as we should be here), kernel density estimation can be used instead.

Kernel density estimation

Kernel density estimation (KDE) works by smoothing over a horizontal bandwidth of data values to obtain an estimation for the density function used to generate the data values.

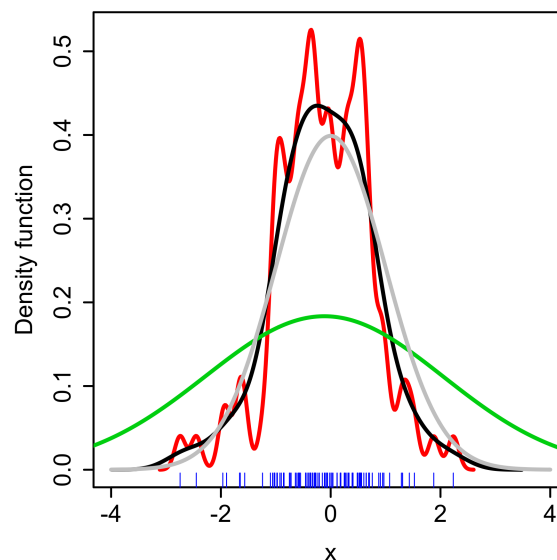
The image on the right shows KDE with different bandwidths of a random sample of 100 points from a standard normal distribution.

Grey: true density (standard normal).

Red: KDE with bandwidth 0.05.

Black: KDE with bandwidth 0.337.

Green: KDE with bandwidth 2.



```
nb_kernel <- naive_bayes(y ~ x1 + x2 + x3, data = heart, usekernel = T)
head(predict(nb_kernel, type = "prob")) |> round(3)
```

```
      0      1
[1,] 0.409 0.591
[2,] 0.243 0.757
[3,] 0.997 0.003
[4,] 0.716 0.284
[5,] 0.807 0.193
[6,] 0.365 0.635
```

```
predict(nb_kernel, type = "class")
```

```
[1] 1 1 0 0 0 1 0 1 1 1 0 1 1
```

```
Levels: 0 1
```

```
heart$y
```

```
[1] 1 1 0 0 0 0 0 1 1 1 0 1 1
```

```
Levels: 0 1
```

```
par(mfrow = c(2, 2))
```

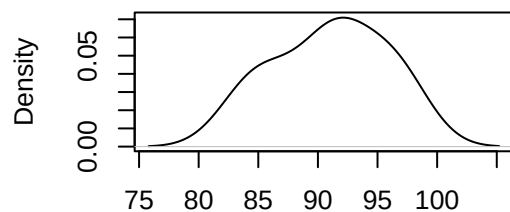
```
plot(density(heart$x1[heart$y == 1]), main = "Density Estimate for X1|Y=1")
```

```
plot(density(heart$x1[heart$y == 0]), main = "Density Estimate for X1|Y=0")
```

```
plot(density(heart$x3[heart$y == 1]), main = "Density Estimate for X3|Y=1")
```

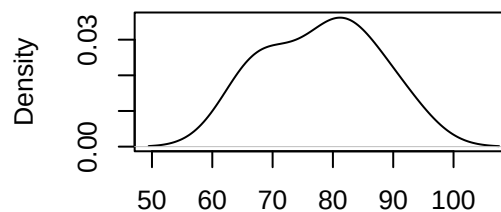
```
plot(density(heart$x3[heart$y == 0]), main = "Density Estimate for X3|Y=0")
```

Density Estimate for X1|Y=1



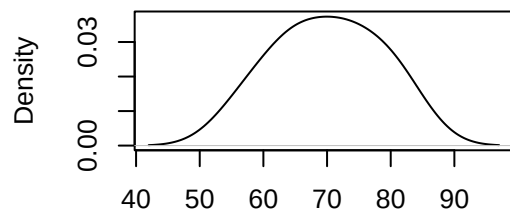
N = 7 Bandwidth = 2.731

Density Estimate for X1|Y=0



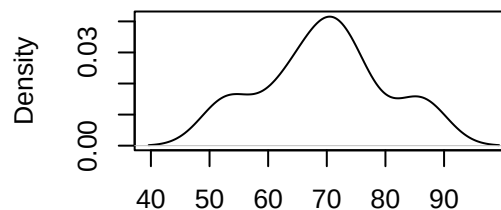
N = 6 Bandwidth = 5.515

Density Estimate for X3|Y=1



N = 7 Bandwidth = 5.006

Density Estimate for X3|Y=0



N = 6 Bandwidth = 4.459

```
par(mfrow = c(1, 1))
```

Advantages and Disadvantages of Naïve Bayes

Advantages:

- **Fast Speed:** Because it treats every variable independently, it doesn't have to perform complex matrix math and can handle massive datasets.
- **High-Dimensionality:** It excels when you have a huge number of features, such as text classification (spam detection), where every word is a feature. Other classifiers may be too slow or crash.
- **Robust to Irrelevant Features:** If you accidentally include irrelevant variables naïve Bayes often filters them out or puts little weight on them.

- **Good with Small Sample Size:** It requires less training data than some other classifiers because it only needs to estimate the distribution of each variable individually.

Disadvantages:

- **The Independence Assumption:** In the real world, variables are almost always related. By ignoring correlations, naïve Bayes can produce a much accurate decision boundary than other classifiers like LDA (seen next).
- **Block Box:** Unlike LDA, where you can easily see which variables are driving the classification, SVM (especially with a Radial Basis kernel) can be hard to interpret. It is difficult to explain why a specific point was classified a certain way.
- **The “Zero Frequency” Problem:** If a specific category or value appears in your test data but was never seen during training, the model assigns it a probability of zero (this can be fixed with “Laplace Smoothing”).
- **Poor Probability Estimates:** It is often good at predicting the correct class, the actual probability scores it gives are often exaggerated and shouldn’t be relied upon for risk assessment.
- **Shape Limitations:** It usually creates linear or simple boundaries. It cannot capture the intricate, curvy relationships that some other classifiers can.

Linear Discriminant Analysis

We just saw with the naïve Bayes classifier that posterior probabilities are computed with the assumption that each feature (predictor) is independent. Another approach that is similar is linear discriminant analysis (LDA). This time, we will have the following assumptions:

1. Each predictor follows a normal (Gaussian) distribution.
2. The variance for each predictor variable is equal across each class of Y .

The way this works, like with naïve Bayes, is by using Bayes' Theorem. We again begin with the formulation we have from naïve Bayes:

$$P(y_i = k | \mathbf{x}_i) = \frac{P(\mathbf{x}_i | y_i = k)P(y_i = k)}{P(\mathbf{x}_i)} = \frac{P(\mathbf{x}_i | y_i = k)P(y_i = k)}{\sum_{\ell=1}^K P(y_i = \ell)P(\mathbf{x}_i | y_i = \ell)}.$$

$P(y_i = k)$ can either come from the proportions of each class in the data or from some prior probability. The main difference between LDA and naïve Bayes is in the assumptions and in how $P(\mathbf{x}_i | y_i = k)$ is handled. We do not assume feature independence here, so we will not write $P(\mathbf{x}_i | y_i = k) = P(x_{i1} | y_i = k) \times \cdots \times P(x_{ip} | y_i = k)$. Let's begin with the case when we have one predictor variable.

LDA for One Predictor

When we only have one predictor variable, X , then $P(X = x_i | y_i = k)$ is modeled as coming from a univariate normal (Gaussian) distribution:

$$P(x_i | y_i = k) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x_i - \mu_k)^2\right)$$

where μ_k is the mean of X when $Y = k$ and σ_k is the standard deviation of X when $Y = k$. Now, in reality, we will have to estimate μ_k and σ_k with $\hat{\mu}_k$ and $\hat{\sigma}_k$.

To find $\hat{\mu}_k$, we take

Finding $\hat{\sigma}_k$ is a little different because we are assuming the variance of the classes is equal. Therefore, we do not think about a different $\hat{\sigma}_k$ for each group. Instead, we think about one $\hat{\sigma}$ that we will use for all groups. That means that practically we would write

$$P(\widehat{x_i | y_i = k}) = \frac{1}{\sqrt{2\pi}\hat{\sigma}} \exp\left(-\frac{1}{2\hat{\sigma}^2}(x_i - \hat{\mu}_k)^2\right),$$

though we often drop the hat over the probability for convenience purposes.

To estimate $\hat{\sigma}$ we need to pool the variances from each class. That gives us

$$\begin{aligned} \hat{\sigma} &= \sqrt{\frac{1}{n-K} \sum_{k=1}^K \left(\sum_{y_i=k} (x_i - \hat{\mu}_k)^2 \right)} \\ &= \sqrt{\frac{1}{n-K} \sum_{k=1}^K (n_k - 1) \cdot \hat{\sigma}_k^2} \end{aligned}$$

where K is the number of classes we have present in the data, n_k is the sample size for class k , and $\hat{\sigma}_k^2$ is the sample variance of the values of X where $Y = k$.

LDA with One Predictor in R

We will use the `lda()` function in the **MASS** library in **R** for this. Notice that **MASS** has some function names that are the same as those in the **tidyverse**, so be careful if using them both. For example, **MASS** has a `select()` function. If you want to use the **tidyverse** `select()` function, indicate that with the name of the package it comes from: `dplyr::select()`.

```
library(MASS)
# First when only one predictor
lda1 <- lda(y ~ x1, data = heart) # heart is same dataset used with naive Bayes
options(pillar.sigfig = 7) # Adjust the number of significant figures to 7
heart |> group_by(y) |> summarize(mean_x1 = mean(x1), sd_x1 = sd(x1), n = n())
# A tibble: 2 x 4
  y      mean_x1    sd_x1      n
  <fct>    <dbl>    <dbl> <int>
1 0          78 9.338094      6
2 1          91 4.795832      7

sigmahat <- sqrt(1/(13 - 2) * ((6 - 1)*9.338094^2 + (7 - 1)*4.795832^2))
heart[1,]
  y x1 x2 x3
1 1 92  F 71

6/13*dnorm(92, 78, sigmahat) / # Predicted prob that y = 0
(6/13*dnorm(92, 78, sigmahat) + 7/13*dnorm(92, 91, sigmahat))
[1] 0.1168486

predict(lda1)$posterior[1,] # Show this matches
      0      1
0.1168486 0.8831514

lda1
Call:
lda(y ~ x1, data = heart)

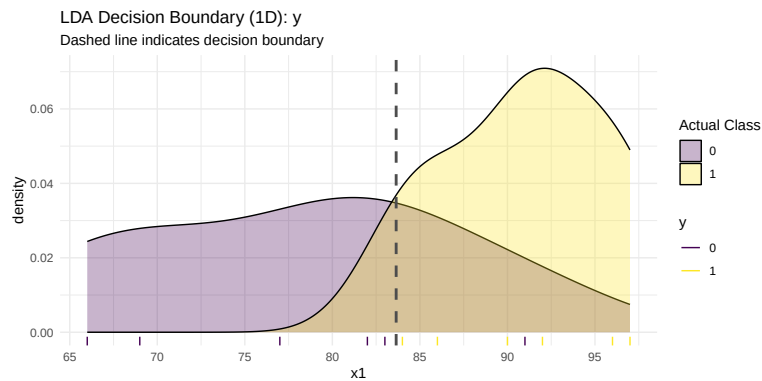
Prior probabilities of groups:
      0      1
0.4615385 0.5384615

Group means:
  x1
0 78
1 91

Coefficients of linear discriminants:
  LD1
x1 0.1384332
```

The “Coefficients of linear discriminants” value (0.1384332 in the first LDA example) is $1/\hat{\sigma}$ when we have only one predictor variable and is not very useful in this case.

```
classifier_plot(lda1, data = heart) + scale_x_continuous(breaks = seq(65, 95, 5))
```



LDA with One Predictor when Y has 3+ Classes

```
# Three categories
lda2 <- lda(y ~ x1, data = heart2)
lda2
Call:
lda(y ~ x1, data = heart2)
```

Prior probabilities of groups:

```
      0      1      2
0.4615385 0.3076923 0.2307692
```

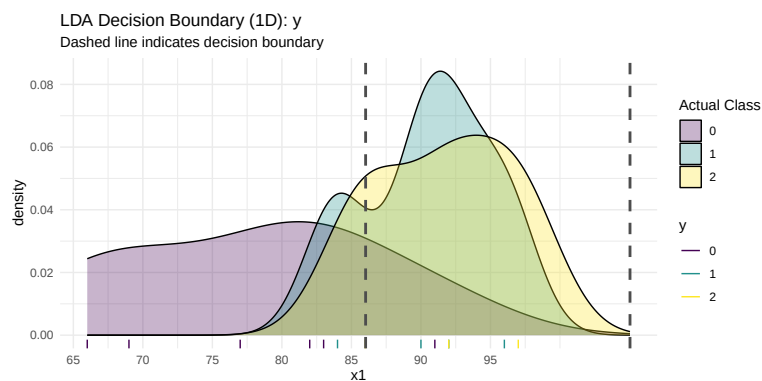
Group means:

```
      x1
0 78.00000
1 90.50000
2 91.66667
```

Coefficients of linear discriminants:

```
LD1
x1 0.13226
```

```
classifier_plot(lda2, data = heart2) + scale_x_continuous(breaks = seq(65, 95, 5))
```



LDA with More than One Predictor

When we have two or more predictor variables (p of them), then we think of $P(\mathbf{x}_i|y_i = k)$ as coming from a multivariate normal (Gaussian) distribution:

$$P(\mathbf{x}_i|y_i = k) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T \Sigma^{-1}(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)\right),$$

where $\mathbf{x}_i$ is i row of the data matrix of X values, $\hat{\boldsymbol{\mu}}_k$ is the vector of sample means for each of the X variables for class k , and Σ is the pooled variance-covariance matrix of the predictor variables found by taking

$$\Sigma = \frac{\sum_{k=1}^K (n_k - 1) \Sigma_k}{n - K},$$

where Σ_k is the variance-covariance matrix of the predictor variables for the k th class. Note that $|\Sigma|$ is the determinant of the matrix Σ .

LDA with Two Predictors in R

```
# Back to the case where we only have two classes
lda3 <- lda(y ~ x1 + x3, data = heart)
heart[1,]
  y x1 x2 x3
1 1 92  F 71

x <- c(92, 71)
Sigma = (
  (6 - 1) * cov(cbind(heart$x1[heart$y == 0], heart$x3[heart$y == 0])) +
  (7 - 1) * cov(cbind(heart$x1[heart$y == 1], heart$x3[heart$y == 1]))
) / (13 - 2)
mu_0 <- c(mean(heart$x1[heart$y == 0]), mean(heart$x3[heart$y == 0]))
mu_1 <- c(mean(heart$x1[heart$y == 1]), mean(heart$x3[heart$y == 1]))

prob0 <- 1/((2*pi)^(2/2)*det(Sigma)^(1/2)) *
  exp(-1/2 * t(x - mu_0) %*% solve(Sigma) %*% (x - mu_0))
prob1 <- 1/((2*pi)^(2/2)*det(Sigma)^(1/2)) *
  exp(-1/2 * t(x - mu_1) %*% solve(Sigma) %*% (x - mu_1))
6/13 * prob0 / (6/13 * prob0 + 7/13 * prob1) # Predicted prob that y = 0
[1,]
[1,] 0.1051156

predict(lda3)$posterior[1,] # Compare to function probability
      0      1
0.1051156 0.8948844

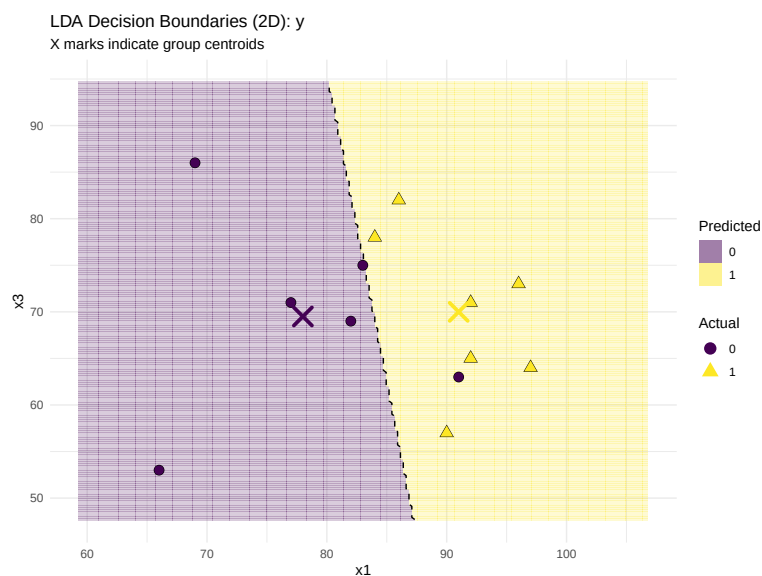
lda3$scaling # Coefficients of linear discriminants
      LD1
x1 0.14061517
x3 0.02111194
```

The “Coefficients of linear discriminants” are now found using $\Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$. These values are now relevant because they tell us two things:

1. The sign of these coefficients tell us if an increase of that variable is attributed to Y being more or less likely to be in the category we are trying to predict (the second category of a factor, by default). Positive values indicate Y is more likely in that class if that X variable increases.
2. The magnitude of one of the values compared to the other tells how much more important that variable is in separating classes than the other.

In this example:

```
classifier_plot(lda3, data = heart)
```



LDA with Multiple Predictors and Three Classes

The idea when we have three classes is very similar to when we have two. The formulas and functions presented earlier don't change. Some of the output and interpretation changes, though.

```
lda4 <- lda(y ~ x1 + x2 + x3, data = heart2)
```

```
lda4
```

```
Call:
```

```
lda(y ~ x1 + x2 + x3, data = heart2)
```

Prior probabilities of groups:

```
      0      1      2
0.4615385 0.3076923 0.2307692
```

Group means:

	x1	x2M	x3
0	78.00000	0.166667	69.50000
1	90.50000	0.750000	69.75000
2	91.66667	0.666667	70.33333

Coefficients of linear discriminants:

	LD1	LD2
x1	0.13149589	0.06424676
x2M	2.00358894	-1.58052486
x3	-0.02159012	0.06305161

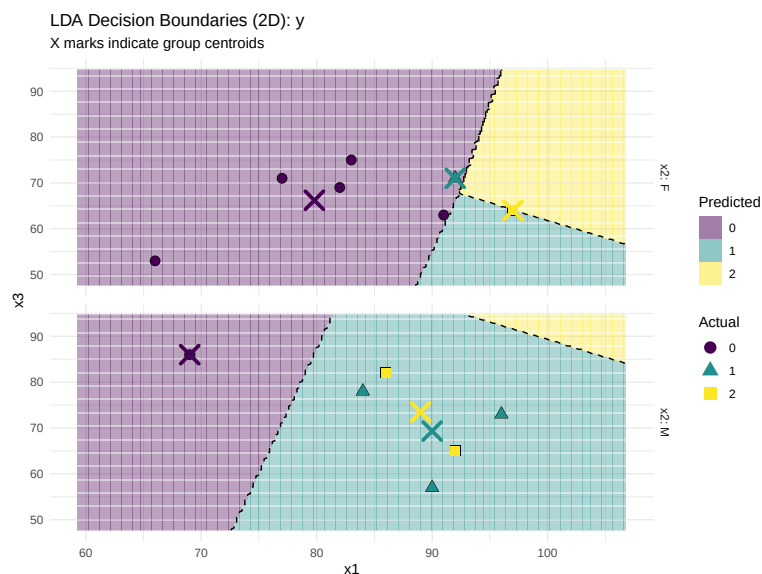
Proportion of trace:

LD1	LD2
0.996	0.004

Now we have LD1 and LD2 each with coefficients for all variables. The first linear discriminant, LD1, is trying to find the maximum possible separation between all group means. You can think of this as the “super-separator”. It is saying that being male (x2) is by far the most useful for that, then diastolic blood pressure (x1), than age (x3).

The second linear discriminant, LD2, is basically the “refiner”. It is mathematically forced to be uncorrelated (perpendicular) to LD1. Its job is to find the next best direction to separate the groups that LD1 couldn’t distinguish. In this case, x2 is still the most useful, but now x3 is about as helpful as x1 in refining since x1 was the more dominant force in initially separating.

Finally, there is a “proportion of trace” measure. This tells us how much separating power comes from each linear discriminant. LD1 will always have a larger proportion of trace than LD2. In this case, LD1’s proportion of trace is 0.996, which means that about 99.6% of the the difference between our three groups can be explained using just the first formula (LD1). The second formula (LD2) only adds 0.4% more information. This is similar to PCA.



Computing Coefficients of Linear Discriminants (3+ Classes)

When there are more than two classes for Y , the “Coefficients of linear discriminants” are found by quantifying the sum of squares between classes (SSB) and the sum of squares within classes (SSW). Then we do something similar to $\Sigma^{-1}(\mu_1 - \mu_0)$. Σ is quantifying uncertainty (like SSW does) and $(\mu_1 - \mu_0)$ is quantifying how different the class means are (like SSB does). So, we will find the matrix for SSW, which we will call $\mathbf{W}$ and the matrix for SSB, which we will call $\mathbf{B}$ and then take $\mathbf{W}^{-1}\mathbf{B}$. However, it does not end there. We then take the eigenvectors of that new matrix and scale them. If we call the eigenvectors $v_1, \dots, v_p$ (there will be one per predictor variable), then the coefficients of linear discriminant for each discriminant is found using

$$w_i = v_i \cdot \sqrt{\frac{n - K}{v_i^T \mathbf{W} v_i}}$$

We can do this in **R** like this:

```
lda4 <- lda(y ~ x1 + x3, data = heart2)
# First quantify W and B using manova (multivariate ANOVA):
W <- summary(manova(cbind(x1, x3) ~ y, data = heart2))$SS$Residuals
B <- summary(manova(cbind(x1, x3) ~ y, data = heart2))$SS$y
# Find eigen vectors
v1 <- eigen(solve(W) %*% B)$vectors[,1]
v2 <- eigen(solve(W) %*% B)$vectors[,2]
v1 %*% sqrt((13 - 3)/(t(v1) %*% W %*% v1)) # Replace v1 with v2 for the other
      [,1]
[1,] 0.13438636
[2,] 0.02046672

lda4$scaling # Compare to the lda function output
      LD1      LD2
x1 0.13438636 -0.003904491
x3 0.02046672  0.096259448
```

Advantages and Disadvantages of LDA

Advantages:

- **Multiclass Handling:** LDA handles multiple classes much better than something like logistic regression.
- **Feature Correlation:** LDA models the relationships between features by estimating a covariance matrix.
- **Dimensionality Reduction:** LDA is unique because it can be used to compress your data into a smaller number of “discriminant coordinates.” This makes it much easier to visualize high-dimensional data in a 2D plot than it is with SVM.
- **Efficiency on Small Datasets:** When your data are normally distributed and you have a small sample size, LDA is often more stable and accurate than some other classifiers like SVM (seen next).

Disadvantages:

- **Strict Statistical Assumptions:** LDA assumes your data follows a Gaussian (normal) distribution and that all classes have equal variance. If your data is messy or non-normal, naïve Bayes or SVM will usually outperform it. Technically, LDA can work without these assumptions, but most software codes this in. To drop the equal variance assumption, quadratic discriminant analysis (QDA) is often used.
- **Sensitivity to Outliers:** LDA relies on the mean of the data, so a single extreme outlier can pull the decision boundary significantly out of place.
- **Linear Limitation:** LDA can only draw straight lines or flat planes. If your classes are separated by a curve or a circle, LDA will fail.
- **Covariance Modeling:** LDA needs to estimate a covariance matrix. If there are a very large number of features but not enough rows of data, this estimation can fail.

Support Vector Machines

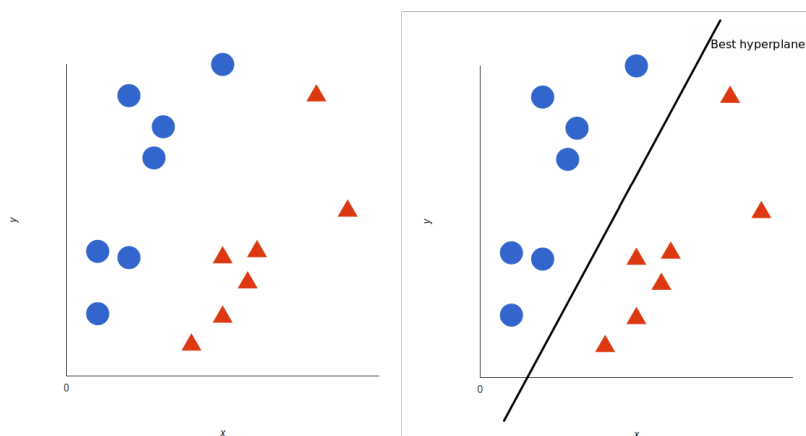
In machine learning, **support vector machines** or **SVMs** are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis. However, they are mostly used in classification problems. Here we will focus on developing intuition rather than rigor and develop a basic understanding of the working principles.

Material for this content was obtained and adapted from:

- <https://www.datacamp.com/community/tutorials/support-vector-machines-r>
- *The Elements of Statistical Learning*, Hastie, et al., Springer
- <https://www.geeksforgeeks.org/classifying-data-using-support-vector-machines-svm-in-r/>

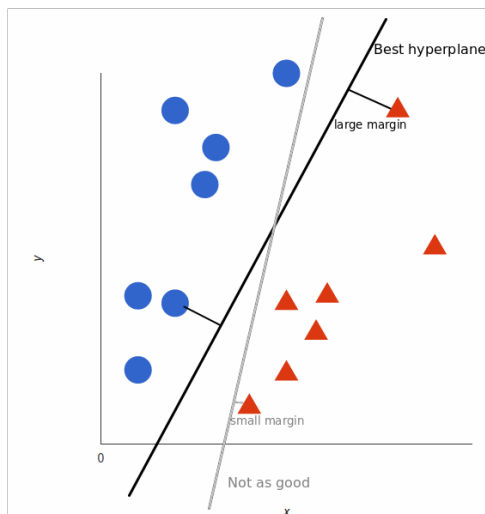
Support Vector Machines—Linear Data

Let's imagine we have two tags: *red* and *blue*, and our data has two features: x and y . We can plot our training data on a plane:



An **SVM** identifies the **decision boundary** or **hyperplane** (a line in two dimensions) that best separates the tags.

But, what exactly is the best hyperplane? For SVM, it's the one that maximizes the margins from the data from both tags:



Types of margins:

There are two types of margins for SVMs: hard margin and soft margin.

- **Hard margin:** This is used when the hyperplane can be drawn in such a way that there are no misclassifications. This is uncommon on real datasets since this would mean the data have no overlap.
- **Soft margin:** This is used when the data are not perfectly separable or contain outliers. Each data point has a slack variable introduced by the soft-margin SVM formulation, which softens the strict margin requirement and permits certain misclassifications or violations. There is a compromise between increasing the margin and reducing violations.

A Look into SVM Methodology

We will focus first on the hard margin case where we can get perfect separation. The hyperplane in 2D is a line. We can think of the equation of this line first in standard form $ax + by = c$. Now, to make this more general, we will write it as $ax_1 + bx_2 - c = 0$. Reparameterizing this, we get

$$\beta_1 x_1 + \beta_2 x_2 - c = 0.$$

Then we can write $x_2 = -\frac{\beta_1}{\beta_2}x_1 + \frac{c}{\beta_2}$. So, the slope of this line is $-\frac{\beta_1}{\beta_2}$ and the y -intercept is $\frac{c}{\beta_2}$.

In k dimensions, we get the equation of our hyperplane as

$$\beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_k x_k - c = 0.$$

this is equivalently written

$$\boldsymbol{\beta}^T \mathbf{x} - c = 0.$$

Hard-Margin Case

Now, in the hard-margin case, we will classify anything on or above the boundary $\boldsymbol{\beta}^T \mathbf{x} - c = 1$ is one class (say a 1) and everything on or below the boundary $\boldsymbol{\beta}^T \mathbf{x} - c = -1$ as the other class (say a -1).

The distance between those two hyperplanes is $\frac{2}{\|\boldsymbol{\beta}\|_2}$ where $\|\boldsymbol{\beta}\|_2$ is the Euclidean norm (or the 2 norm): $\|\boldsymbol{\beta}\|_2 = \sqrt{\sum_{i=1}^k \beta_i^2}$. To maximize the distance between the hyperplanes, that is, to maximize the margin, we want to minimize $\|\boldsymbol{\beta}\|_2$.

We also need to prevent any point from falling in the margin, so we need $\boldsymbol{\beta}^T \mathbf{x}_i - c \geq 1$ if $y_i = 1$ and $\boldsymbol{\beta}^T \mathbf{x}_i - c \leq -1$ if $y_i = -1$. We can combine these constraints by writing $y_i(\boldsymbol{\beta}^T \mathbf{x}_i - c) \geq 1$ for all $1 \leq i \leq n$ and for $y_i \in \{-1, 1\}$. Therefore, finding the β_i values is the same as the constrained optimization problem

$$\begin{aligned} & \underset{\boldsymbol{\beta}, c}{\text{minimize}} \quad \|\boldsymbol{\beta}\|_2^2 \\ & \text{subject to} \quad y_i(\boldsymbol{\beta}^T \mathbf{x}_i - c) \geq 1 \text{ for all } i \end{aligned}$$

An important property of this optimization problem is that the max-margin hyperplane is completely determined by those $\mathbf{x}_i$ that lie nearest to it. These $\mathbf{x}_i$ are called **support vectors**.

Soft-Margin Case

Much remains the same in the soft-margin case except now it is not possible to prevent values from falling in the margin. So, we introduce new variables, ξ_i that are equal to zero for points on the correct side of the margin and equal to the distance to the end of the margin for points on the wrong side of the margin. See Figure 12.1 below, which comes from *Elements of Statistical Learning*:

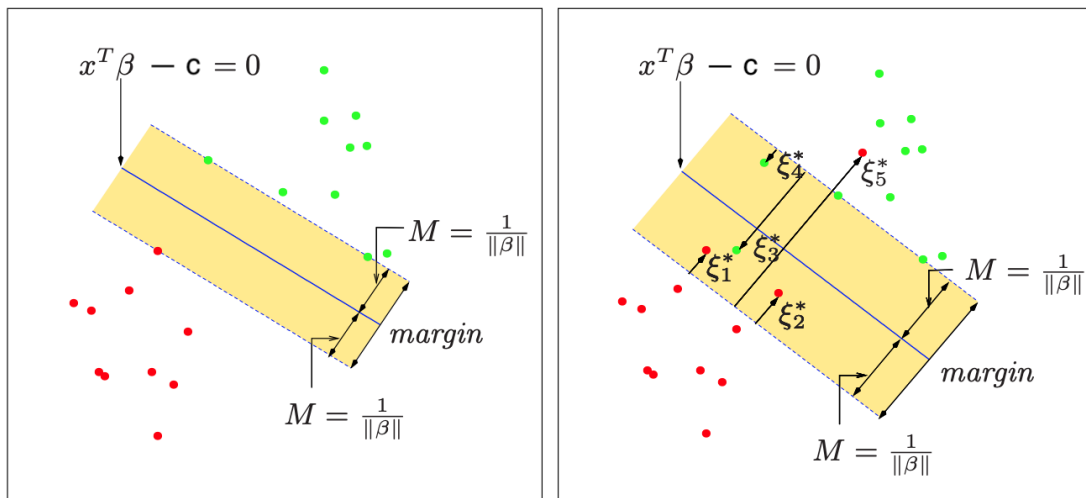


FIGURE 12.1. Support vector classifiers. The left panel shows the separable case. The decision boundary is the solid line, while broken lines bound the shaded maximal margin of width $2M = 2/\|\beta\|$. The right panel shows the nonseparable (overlap) case. The points labeled ξ_j^* are on the wrong side of their margin by an amount $\xi_j^* = M\xi_j$; points on the correct side have $\xi_j^* = 0$. The margin is maximized subject to a total budget $\sum \xi_i \leq \text{constant}$. Hence $\sum \xi_j^*$ is the total distance of points on the wrong side of their margin.

Therefore, the new optimization problem can be stated

$$\begin{aligned} & \underset{\beta, c, \xi}{\text{minimize}} \quad \|\beta\|_2^2 + C \sum_{i=1}^n \xi_i \\ & \text{subject to} \quad y_i(\beta^T \mathbf{x}_i - c) \geq 1 - \xi_i \text{ where } \xi_i \geq 0 \text{ for all } i \end{aligned}$$

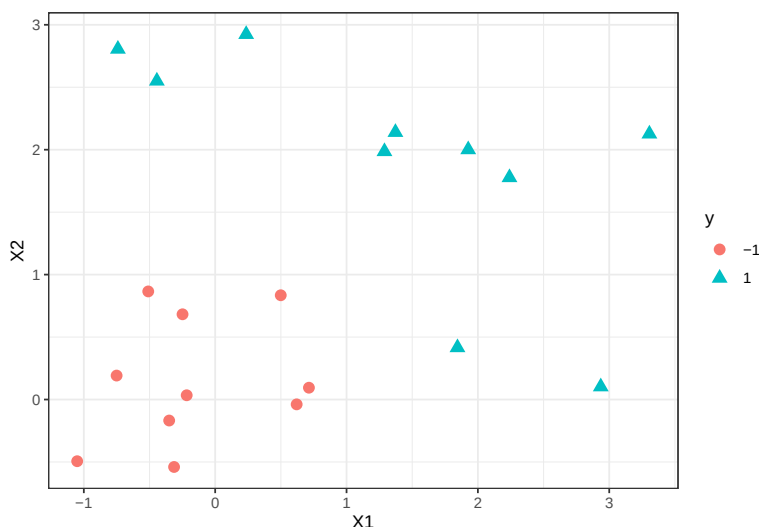
where $C > 0$ is a cost parameter that determines the trade-off between increasing the margin size and ensuring that point i lie on the correct side of the margin. The smaller the cost, the more misclassifications are allowed in the data and the more potential to under fit. The higher the cost, the fewer misclassifications and the more potential to over fit. The higher the cost, the more this behaves like the hard-margin case. Using cross validation may be a good way to choose this cost value.

Support Vector Machines in R

Hard-margin Case

First generate some data in 2 dimensions, and make them a little separated:

```
set.seed(10111)
x = matrix(rnorm(40), 20, 2)
y = rep(c(-1, 1), c(10, 10))
x[y == 1,] = x[y == 1,] + 1.6 # Add separation to the classes
dat <- data.frame(X1 = x[,1], X2 = x[,2], y = as.factor(y))
ggplot(dat, aes(x = X1, y = X2, color = y, shape = y)) +
  geom_point(size = 3) +
  theme_bw()
```



We will use the **e1071** package which contains the `svm` function and make a dataframe of the data, turning `y` into a factor variable. The `cost` option in the `svm()` function controls training errors and margins.

```
library(e1071)
# The variable entered first will be plot on the y axis
svmfit <- svm(y ~ X1 + X2, data = dat,
              kernel = "linear", cost = 10, scale = FALSE)
print(svmfit)
```

Call:

```
svm(formula = y ~ X1 + X2, data = dat, kernel = "linear", cost = 10,
     scale = FALSE)
```

Parameters:

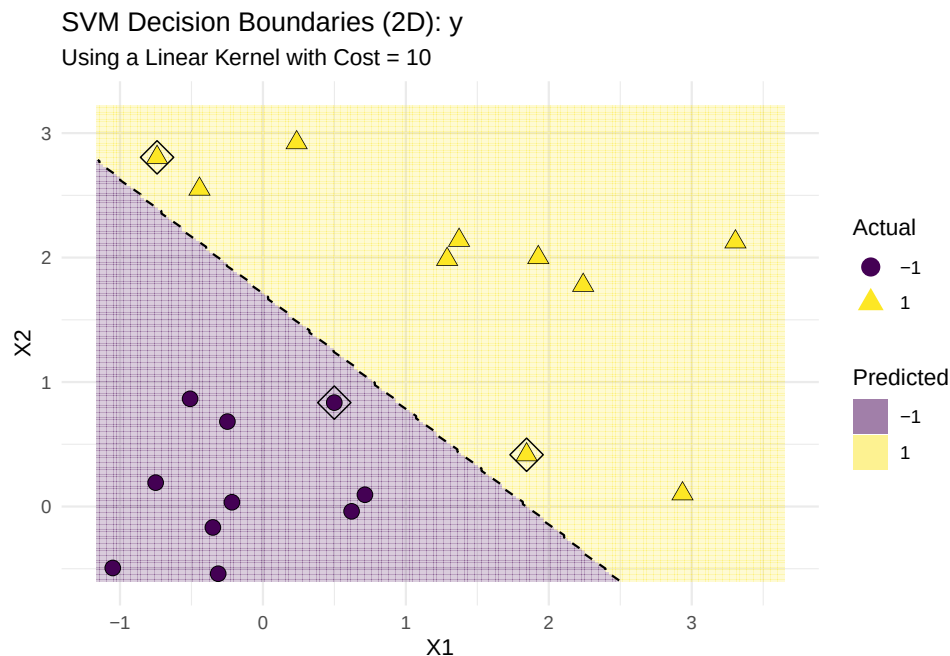
```
SVM-Type:  C-classification
SVM-Kernel: linear
cost:      10
```

Number of Support Vectors: 3

Printing the `svmfit` gives its summary. You can see that the number of support vectors is 3 - they are the points that are close to the boundary or on the wrong side of the boundary.

There's a plot function for SVM that shows the decision boundary, as you can see below. The points plotted with diamonds around them are support vectors.

```
classifier_plot(svmfit, data = dat)
```



Unfortunately, the `svm()` function is not super friendly. We have to do some work to get back the linear coefficients. The reason is probably that this only makes sense for linear kernels, and the function is more general. So let's use some coded to extract the coefficients. We can extract β and c , which are the linear coefficients using the following:

```
(beta = drop(t(svmfit$coefs)%*%x[svmfit$index,]))
```

```
[1] -2.240242 -2.423619
```

```
(c = svmfit$rho)
```

```
[1] -4.140447
```

Now we can replot the points on the grid, then put the points back in (including the support vector points). Then you can use the coefficients to draw the decision boundary using a simple equation of the form:

$$x_1\beta_1 + x_2\beta_2 - c = 0$$

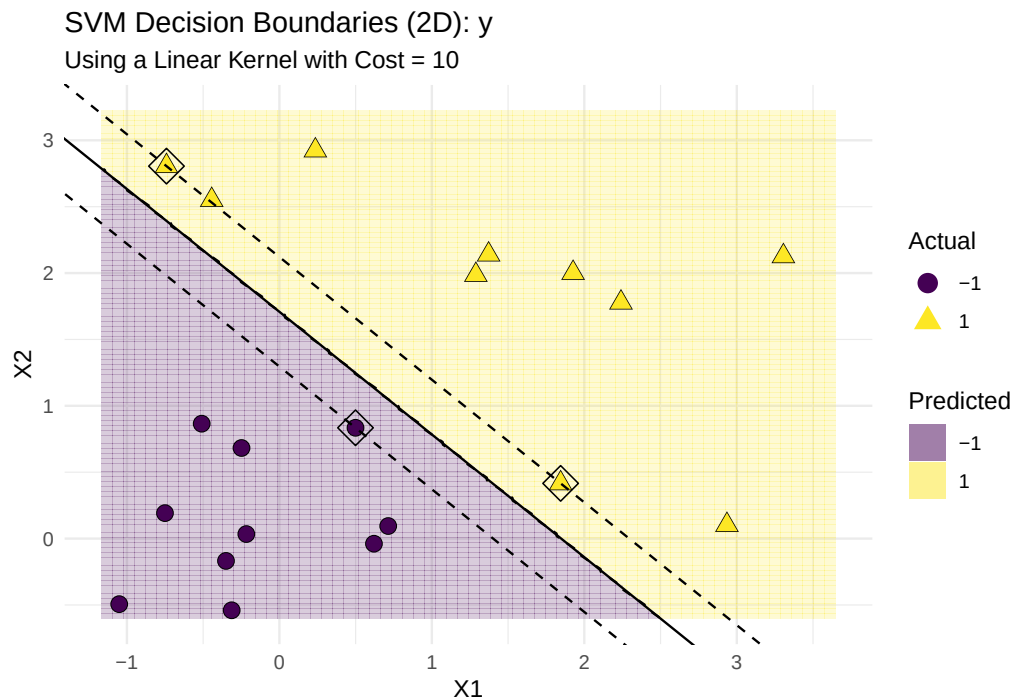
Solving for x_2 gives us

$$x_2 = \frac{c}{\beta_2} - \frac{\beta_1}{\beta_2}x_1$$

Now plotting the line on a graph:

```
classifier_plot(svmfit, data = dat) +  
  geom_abline(intercept = c / beta[2],  
             slope = -beta[1] / beta[2]) +  
  geom_abline(intercept = (c - 1) / beta[2],  
             slope = -beta[1] / beta[2], linetype = "dashed") +
```

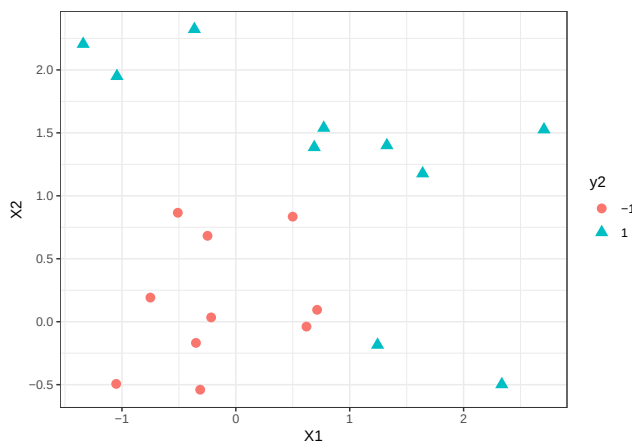
```
geom_abline(intercept = (c + 1) / beta[2],
            slope = -beta[1] / beta[2], linetype = "dashed")
```



Soft-margin Case

Now generate similar data in 2 dimensions, but make them less separated:

```
set.seed(10111)
x2 = matrix(rnorm(40), 20, 2)
y2 = rep(c(-1, 1), c(10, 10))
x2[y2 == 1,] = x2[y2 == 1,] + 1
dat2 <- data.frame(X1 = x2[,1], X2 = x2[,2], y2 = as.factor(y2))
ggplot(dat2, aes(x = X1, y = X2, color = y2, shape = y2)) +
  geom_point(size = 3) +
  theme_bw()
```



```
# The variable entered first will be plot on the y axis
svmfit2 <- svm(y2 ~ X1 + X2, data = dat2,
               kernel = "linear", cost = 10, scale = FALSE)
print(svmfit2)
```

Call:

```
svm(formula = y2 ~ X1 + X2, data = dat2, kernel = "linear", cost = 10,
     scale = FALSE)
```

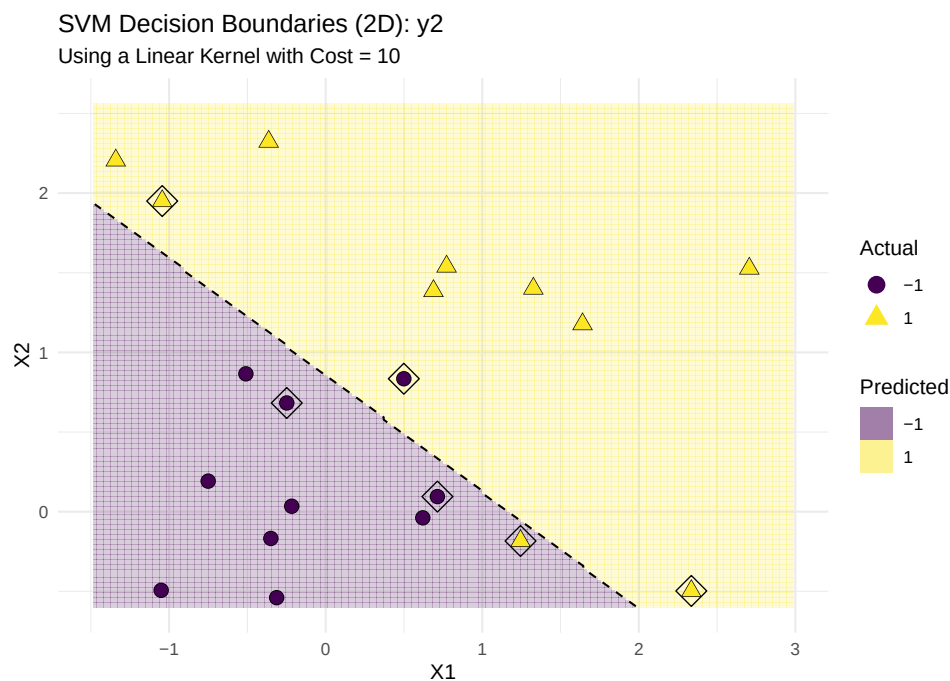
Parameters:

```
SVM-Type: C-classification
SVM-Kernel: linear
cost: 10
```

Number of Support Vectors: 6

Now the number of support vectors is 6 - they are the points that are close to the boundary or on the wrong side of the boundary.

```
classifier_plot(svmfit2, dat2)
```

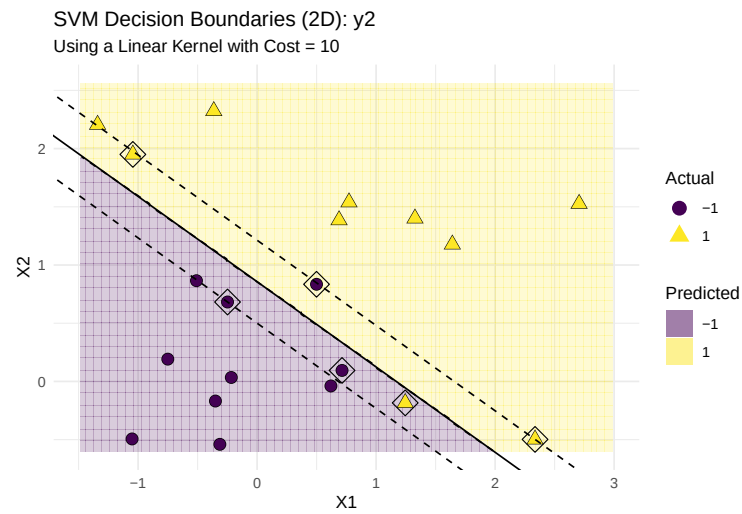


Obtain the coefficients:

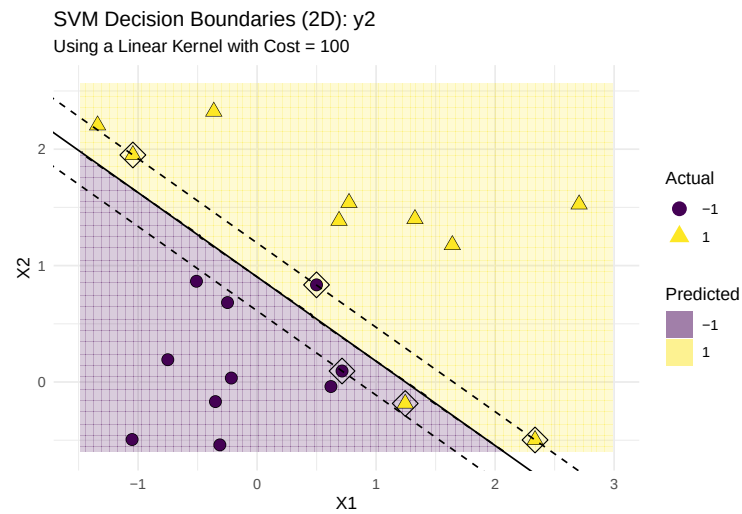
```
(beta = drop(t(svmfit2$coefs)%*%x2[svmfit2$index,]))
[1] -2.055647 -2.806278
```

```
(c = svmfit2$rho)
[1] -2.403917
```

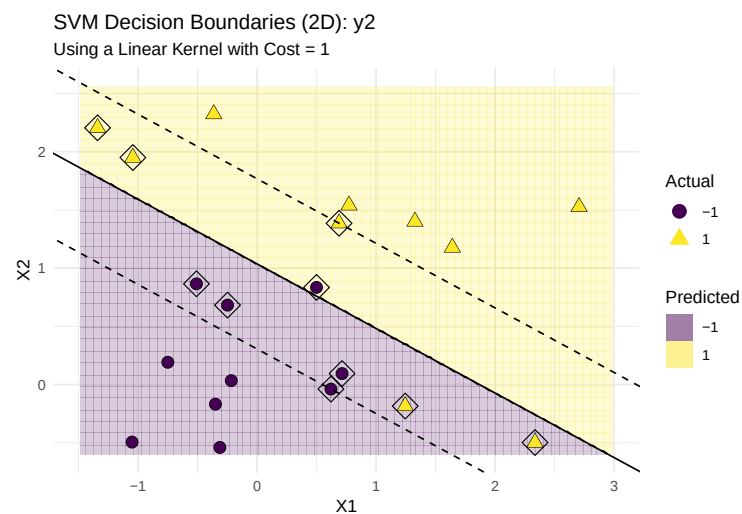
Plot the lines:



If the cost was 100 instead of 10:

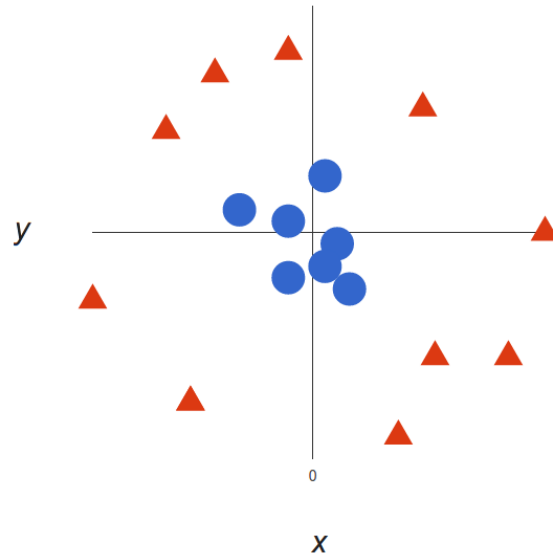


If the cost was 1 instead of 10:



Support Vector Machines for Non-Linear Data

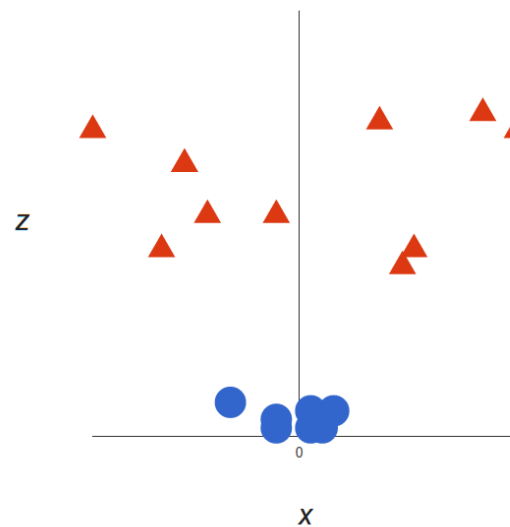
Now the examples before were easy since clearly, the data was linearly separable. Often things aren't that simple. Take a look at this case:



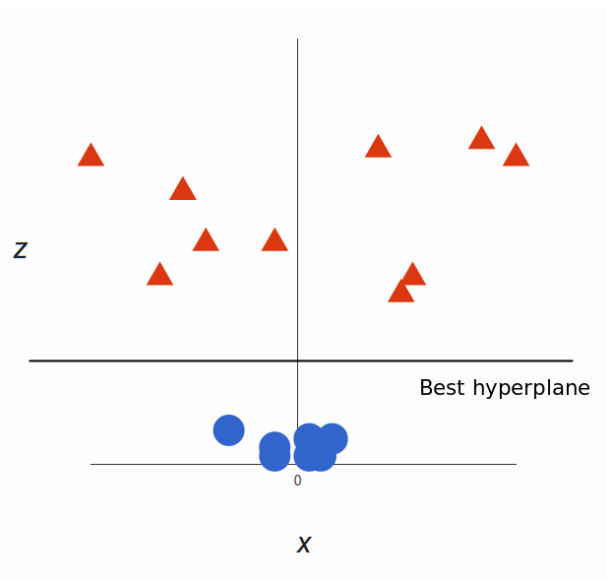
It's pretty clear that there's not a linear decision boundary (a single straight line that separates both tags). However, the vectors are very clearly segregated, and it looks as though it should be easy to separate them.

So here's what we'll do: we will **add a third dimension**. Up until now, we had two dimensions: x and y . We create a new z dimension, and we rule that it be calculated a certain way that is convenient for us: $z = x^2 + y^2$ (you'll notice that's the equation for a circle).

This will give us a three-dimensional space. Taking a slice of that space:

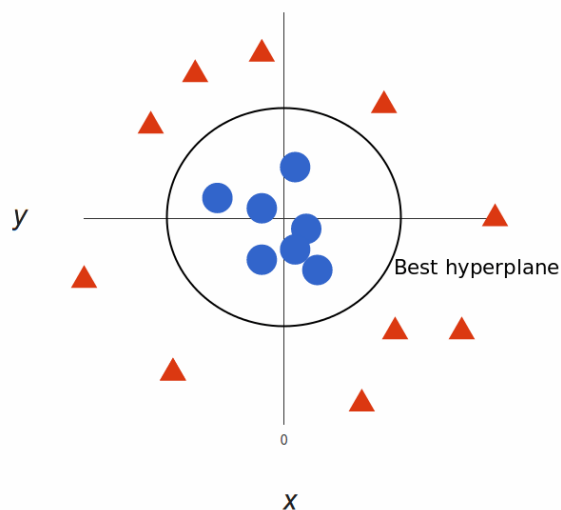


What can SVM do with this? Let's see:



That's great! Note that since we are in three dimensions now, the hyperplane is a plane parallel to the x axis at a certain z (let's say $z=1$).

What's left is mapping it back to two dimensions:



And there we go! Our decision boundary is a circumference of radius 1, which separates both tags using SVM.

Kernel Trick

In this example, we found a way to classify nonlinear data by cleverly mapping our space to a higher dimension. However, it turns out that calculating this transformation can get pretty computationally expensive: there can be a lot of new dimensions, each one of them possibly involving a complicated calculation. Doing this for every vector in the dataset can be a lot of work, so it'd be great if we could find a cheaper solution.

Here's a trick: SVM doesn't need the actual vectors to work its magic, it actually can get by only

with the dot products between them. This means that we can sidestep the expensive calculations of the new dimensions! This is what we do instead:

- Imagine the new space we want:

$$z = x^2 + y^2$$

- Figure out what the dot product in that space looks like. For $\mathbf{a} = [x_a, y_a, z_a]^T$ and $\mathbf{b} = [x_b, y_b, z_b]^T$:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = x_a \cdot x_b + y_a \cdot y_b + z_a \cdot z_b = x_a \cdot x_b + y_a \cdot y_b + (x_a^2 + y_a^2) \cdot (x_b^2 + y_b^2)$$

- Tell SVM to do its thing, but using the new dot product—we call this a **kernel function**.

This is often called the **kernel trick**, which enlarges the feature space in order to accommodate a non-linear boundary between the classes.

Common types of kernels used for SVM are:

- Linear: $\beta^T \mathbf{x} - c$
 - The linear kernel is fast and easy to implement, but it may not capture the complexity and non-linearity of the data. The linear kernel is suitable for high-dimensional and sparse data, where adding more features may cause overfitting or computational problems.
- Polynomial: $(\gamma \beta^T \mathbf{x} - c)^d$. This allows some curvature in the decision boundary.
 - The polynomial kernel is suitable for low-dimensional and dense data, where adding more features may improve the performance and accuracy of the SVM algorithm.
- Radial: $\exp(-\gamma \|\beta - \mathbf{x}\|^2)$. This allows for a lot of curvature in the decision boundary.
 - The radial kernel is suitable for medium-dimensional and diverse data, where the data points may have different scales and distributions. The radial kernel is also known as the Gaussian kernel or the exponential kernel.
- Sigmoid: $\tanh(\gamma \beta^T \mathbf{x} - c)$. Similar to radial, but the curvature behaves differently.
 - The sigmoid kernel is suitable for binary and categorical data, where the data points may have discrete and logical values.

Simply, these kernels transform our data to pass a linear hyperplane and thus classify our data.

Non-linear Support Vector Machines in R

Now let's apply a non-linear (polynomial) SVM to our prior simulated dataset.

```
svmfit_poly = svm(y2 ~ X1 + X2, data = dat2,
                  kernel = "polynomial", cost = 10, scale = FALSE, degree = 3)
print(svmfit_poly)
```

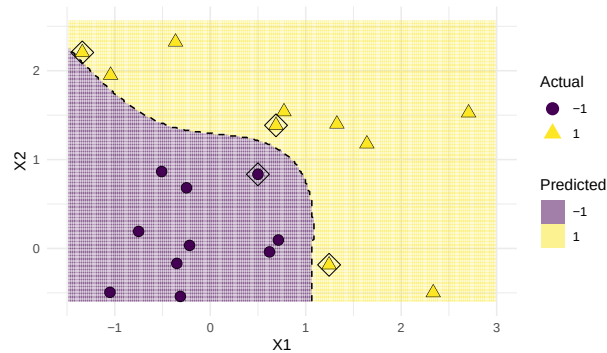
Parameters:

```
SVM-Type: C-classification
SVM-Kernel: polynomial
cost: 10
degree: 4
coef.0: 0
```

Number of Support Vectors: 5

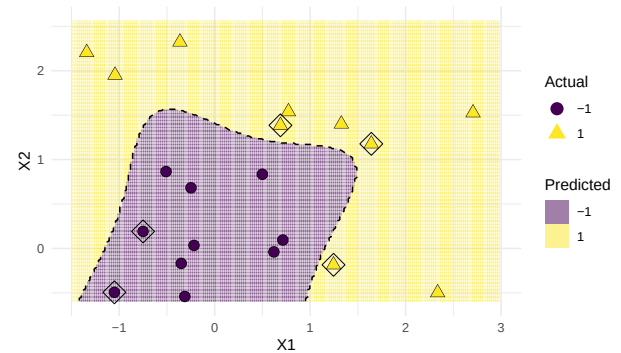
Degree 3 polynomial kernel

SVM Decision Boundaries (2D): y2
Using a Polynomial Kernel with Degree = 3 and Cost = 10



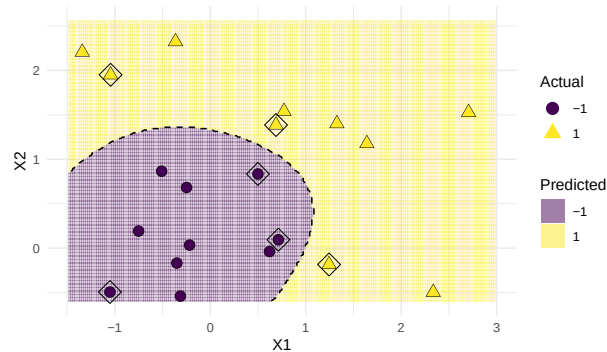
Degree 4 polynomial kernel

SVM Decision Boundaries (2D): y2
Using a Polynomial Kernel with Degree = 4 and Cost = 10



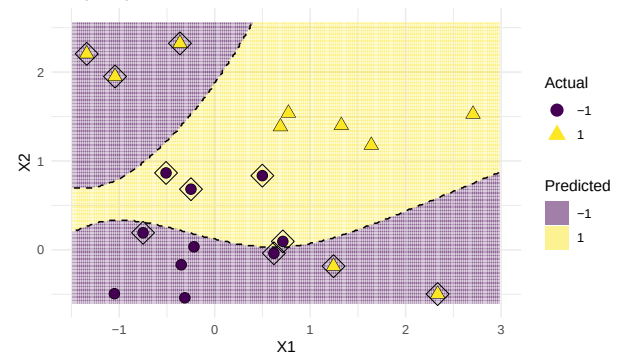
Using a radial kernel:

SVM Decision Boundaries (2D): y2
Using a Radial Basis Kernel with gamma = 0.5 and Cost = 10



Using a sigmoid kernel (not good in this case):

SVM Decision Boundaries (2D): y2
Using a Sigmoid Kernel with Cost = 10

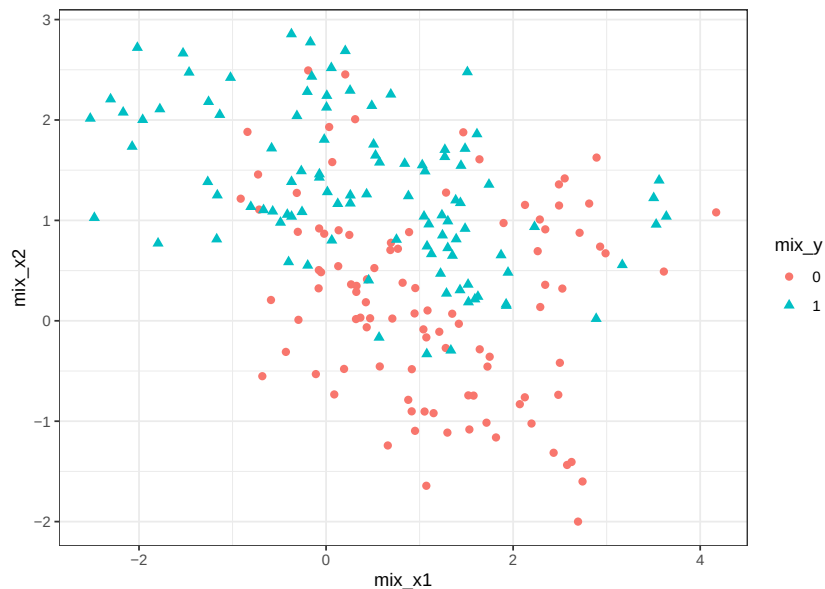


Here is a more complicated example from *Elements of Statistical Learning*, where the decision boundary needs to be non-linear and there is no clear separation. We will use the `ESL.mixture` data object in `math3190package`.

```
library(math3190package) |> suppressPackageStartupMessages()
mix_df <- data.frame(mix_x1 = ESL.mixture$x[,1],
                     mix_x2 = ESL.mixture$x[,2],
                     mix_y = factor(ESL.mixture$y))
```

Plotting the data:

```
ggplot(mix_df,
  aes(x = mix_x1,
      y = mix_x2,
      color = mix_y,
      shape = mix_y)) +
  geom_point(size = 2) +
  theme_bw()
```

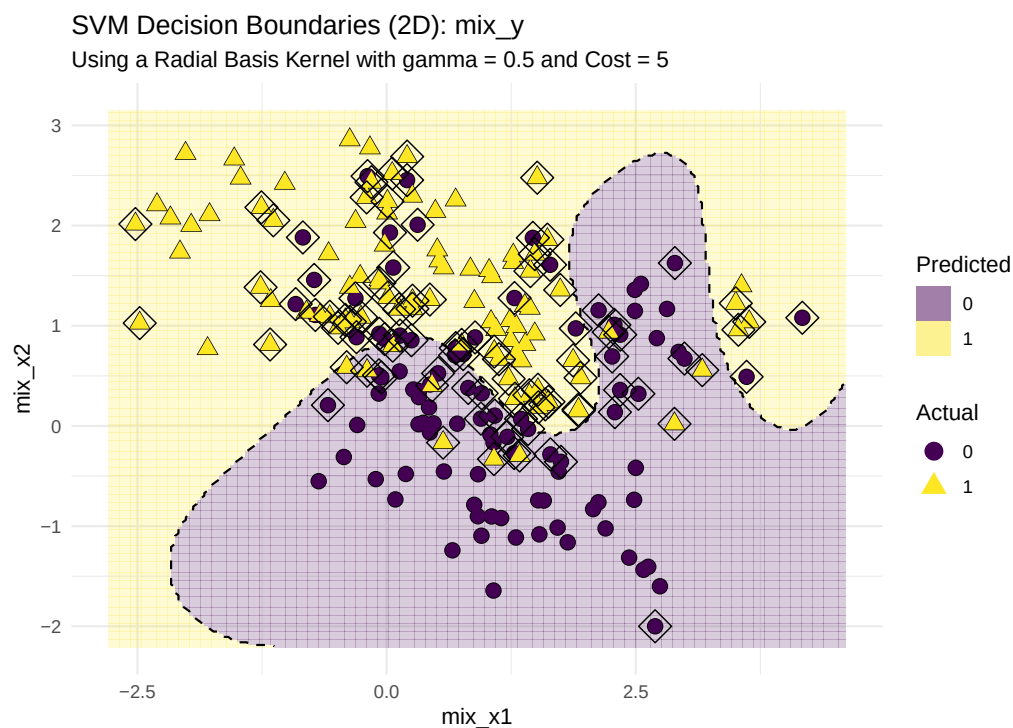


We will fit an SVM with radial kernel.

```
fit_mix <- svm(mix_y ~ mix_x1 + mix_x2, data = mix_df, scale = FALSE,
  kernel = "radial", cost = 5)
```

Plotting the boundary:

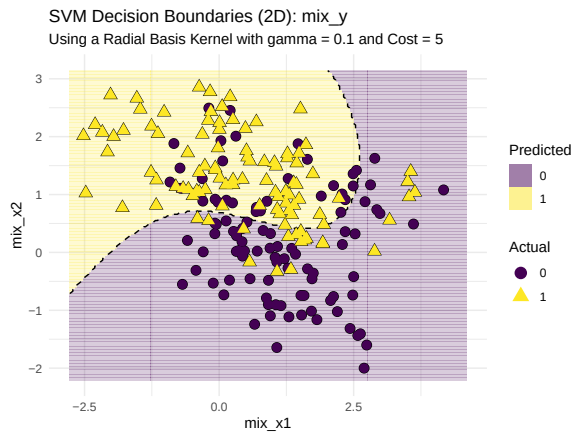
```
classifier_plot(fit_mix, data = mix_df)
```



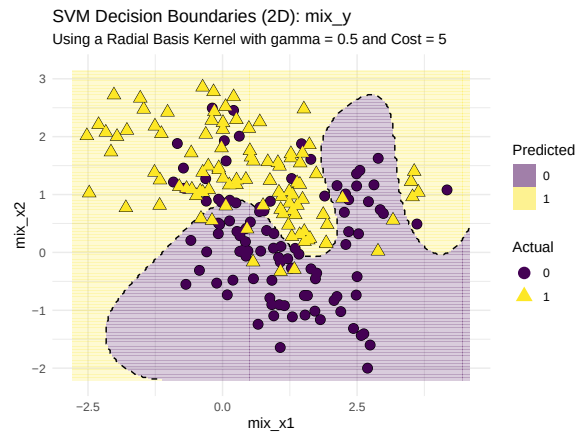
Notice that the response variable labels are 0 and 1 here instead of -1 and 1 .

Let's see the effect of adjusting γ . By default, $\gamma = 1/2$. Let's adjust it to $1/10$, 1 and 2 . The support vectors are not put in diamonds here to avoid too much clutter. There is a `sv_diamonds` argument in the `classifier_plot()` function that can be assigned `FALSE`.

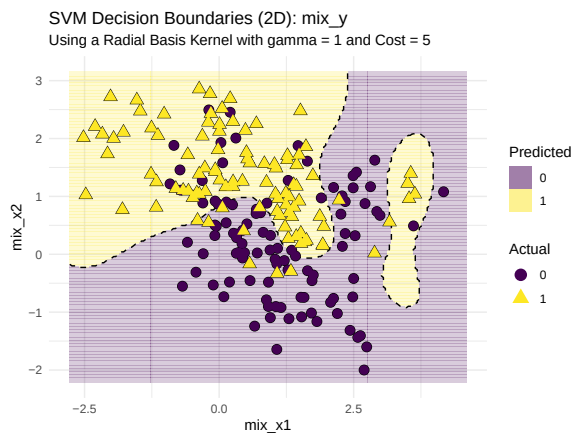
$\gamma = 1/10$:



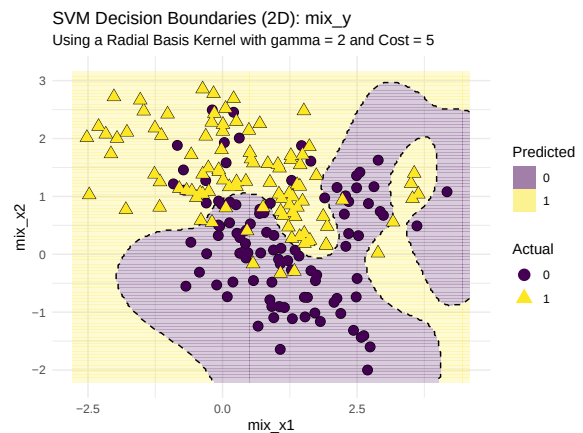
$\gamma = 1/2$:



$\gamma = 1$:



$\gamma = 2$:



How can we choose the best value of γ ?

Advantages and Disadvantages of SVMs

Advantages:

- **High Dimensionality:** SVM is an effective tool in high-dimensional spaces, which is particularly applicable to document classification and sentiment analysis where the dimensionality can be extremely large.
- **Versatility:** Class separation is often highly non-linear. The ability to apply new kernels allows substantial flexibility for the decision boundaries, leading to greater classification performance.
- **No Distributional Assumptions:** SVM does not assume the data follow any particular distribution, making it a non-parameter classifier.

Disadvantages:

- **Kernel Selection:** SVMs are very sensitive to the choice of the kernel parameters. In situations where the number of features for each object exceeds the number of training data samples, SVMs can perform poorly. This can be seen intuitively as if the feature space is much larger than the samples. Then there are less effective support vectors on which to support the optimal linear hyperplanes.
- **Block Box:** Unlike LDA, where you can easily see which variables are driving the classification, SVM (especially with a Radial Basis kernel) can be hard to interpret. It is difficult to explain why a specific point was classified a certain way.
- **Sensitive to Hyperparameters:** To get a good SVM, you have to carefully tune the cost (C) and gamma parameters. A bad choice can lead to severe over fitting.
- **Non-Probabilistic:** Since the classifier works by placing objects above and below a classifying hyperplane, there is no direct probabilistic interpretation for group membership. However, one potential metric to determine the “effectiveness” of the classification is how far from the decision boundary the new point is. However, there are post-processes that can be done to assign probabilities.